TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 7

Python environment, FastAPI install, hello world, params

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. Module 03 — FastAPI from Scratch: overview
2. 03.00 Prerequisites Check
3. 03.01 Python Virtual Environment
4. 03.02 Installing FastAPI
5. 03.03 Hello World
6. 03.04 Path and Query Params

Module 03 — FastAPI from Scratch

What you'll be able to do after this module

- Create and activate a Python virtual environment with `python3 -m venv` and pin project dependencies in a `requirements.txt`
- Build FastAPI routes (GET, POST, DELETE) that use typed path parameters, query parameters, and JSON request bodies
- Validate request data with Pydantic models and interpret FastAPI's automatic 422 validation errors
- Organize a growing application into per-resource routers using `APIRouter` prefixes and tags
- Implement shared logic such as header-based authentication and database sessions with `Depends` dependency injection
- Containerize the app with a Dockerfile and run it alongside Postgres using Docker Compose with healthchecks and named volumes
- Persist data to Postgres using SQLAlchemy ORM models, an engine, and per-request sessions
- Return correct HTTP status codes with `HTTPException` and verify every endpoint with both `curl` and the `/docs` Swagger UI

Lessons and objectives

#	Lesson	Learning objective
00	Prerequisites Check	Verify that Python 3, pip, Docker (without sudo), Docker Compose, curl, and the course directory from Modules 01 and 02 all still work before starting FastAPI.
01	Python Virtual Environment	Create, activate, and deactivate a project-scoped virtual environment with `python3 -m venv` and explain when to use a venv on the host versus running in Docker.
02	Installing FastAPI	Install FastAPI and Uvicorn into the active venv and freeze the exact package versions into a `requirements.txt` with `pip freeze`.
03	Hello World	Write a minimal FastAPI app in `main.py`, run it with `uvicorn main:app --reload`, and verify responses from curl, a browser, and the `/docs` Swagger UI.
04	Path and Query Params	Define routes with typed path parameters and optional query parameters, and demonstrate FastAPI rejecting mistyped input with 422 validation errors.
05	Request Bodies with Pydantic	Validate JSON request bodies on a `POST /items` route with a Pydantic `BaseModel` and test valid and invalid payloads from both curl and Swagger UI.
06	Multiple Routes and Routers	Refactor routes into per-resource modules using `APIRouter` with prefixes and tags, and mount them in `main.py` with `app.include_router`.
07	Dependencies and Injection	Implement a header-checking `get_current_user` dependency with `Depends` and apply it at route level and router level, distinguishing 401 and 422 responses.
08	Containerizing FastAPI	Write a Dockerfile that caches the pip-install layer, build the image, and run the containerized app with uvicorn bound to `0.0.0.0` so the mapped port is reachable.
09	Docker Compose and Postgres	Orchestrate the FastAPI app and a Postgres database with `docker-compose.yml`, using a healthcheck, a named volume, a bind-mounted hot-reload dev loop, and a `.env` file for secrets.
10	Database with SQLAlchemy	Replace the in-memory item store with Postgres persistence using a SQLAlchemy engine, ORM models, and a `get_db` session dependency, then verify the rows directly in psql.
11	Error Handling and Status Codes	Return correct HTTP status codes (201, 204, 400, 404) by setting `status_code` on route decorators and raising `HTTPException` for missing resources and business-rule violations.

#	Lesson	Learning objective
12	Final Checkpoint	Build a complete `Profile` resource from memory — SQLAlchemy model, router, protected DELETE, and correct status codes — and pass a 10-step curl acceptance test.

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

Learning objective: Verify that Python 3, pip, Docker (without sudo), Docker Compose, curl, and the course directory from Modules 01 and 02 all still work before starting FastAPI.

Lesson 00: Prerequisites Check

By the end, you will have confirmed that everything from Modules 01 and 02 still works on your machine.

~10 minutes

---

1. Open VS Code connected to your Ubuntu environment (WSL or SSH from Module 00). Open the integrated terminal.

2. Confirm Python 3 is installed.

```
python3 --version
```

```
Python 3.10.12
```

(Any 3.10+ is fine.)

3. Confirm pip is installed.

```
pip3 --version
```

```
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python 3.10)
```

4. Confirm Docker is installed and you're in the docker group.

```
docker --version
```

```
Docker version 24.0.7, build afdd53b
```

5. Confirm Docker works without sudo.

```
docker run --rm hello-world | head -n 1
```

```
Hello from Docker!
```

6. Confirm Docker Compose is available.

```
docker compose version
```

```
Docker Compose version v2.21.0
```

7. Confirm curl is installed (Module 01 lesson 5 installed it).

```
curl --version | head -n 1
```

```
curl 7.81.0 (x86_64-pc-linux-gnu) ...
```

8. Confirm your course directory exists from Module 01.

```
ls ~/docker-course/
```

You should see `notes`, `practice`, `projects`, etc.

9. Confirm your notes file is intact.

```
tail -n 3 ~/docker-course/notes/notes.txt
```

You should see entries from Modules 01 and 02.

10. Log this check.

```
echo "Module 03 prereq check: passed $(date -Iseconds)" >> ~/docker-
course/notes/notes.txt
```

Watch out

If **any** step above failed, do not continue. Go back and fix it:

- `python3` not found → `sudo apt install -y python3`
- `pip3` not found → `sudo apt install -y python3-pip`

- `docker` permission denied → you skipped `sudo usermod -aG docker $USER && newgrp docker` in Module 02 lesson 01
- `~/docker-course/` missing → you skipped Module 01

Checkpoint

```
python3 --version && pip3 --version && docker --version && docker compose version
&& curl --version | head -n 1
```

You should see five version lines, no errors.

```
docker run --rm hello-world > /dev/null && echo "docker OK"
```

You should see `docker OK`.

```
ls ~/docker-course/notes/notes.txt
```

You should see the path printed.

Next up: 01-python-environment.md — Set up a Python virtual environment for the FastAPI project

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Create, activate, and deactivate a project-scoped virtual environment with `python3 -m venv` and explain when to use a venv on the host versus running in Docker.

Lesson 01: Python Virtual Environment

By the end, you will have a Python virtual environment activated in your project directory and you will know how to toggle it on and off.

~15 minutes of typing

> A **virtual environment** (venv) is an isolated copy of Python and its packages, scoped to one project. Installing FastAPI into a venv keeps it from polluting your system Python.

When to use a venv (especially without Docker)

You use a venv whenever you want a **project-scoped Python** on your machine: predictable package versions, no clashes with system tools, and a clear "this folder owns my dependencies" story. That applies to everyday coding **and** to running checks locally.

Testing and iterating without Docker is one of the main reasons to set this up first:

- **Speed:** Activating a venv and running `pytest`, `uvicorn`, or a one-off script is seconds. Building an image, starting a container, and mounting volumes is slower—fine for integration work, heavy for a tight edit–test loop.
- **IDE and debugger:** Your editor's Python interpreter can point at `.venv`. Breakpoints, "run the test under the caret," and type-checking against installed packages work cleanly without wiring Docker into the IDE (possible, but more moving parts).
- **Failures are easier to read:** Tracebacks and log lines reference paths on your host. When something breaks in a container, you often need to map container paths and env vars back to your repo—again, great for realism, not ideal for the first pass at a failing test.
- **Network and filesystem:** Hitting `localhost:8000` from a browser on your host, reading a local SQLite file, or using a test double does not require publishing ports or bind-mount conventions from day one.

When Docker still matters (later in the course): Containers give you **repeatable environments**—same OS-ish layer, same Python base image, same install steps—so "works on my machine" disappears for deploys and CI. You will often **develop in a venv** and **verify in Docker** (or run CI in Docker) so you get fast local feedback *and* confidence that the app behaves the same when packaged.

Practical split:

Goal	Typical choice
Unit tests, quick API smoke tests, learning FastAPI	Venv on the host (this lesson)
Postgres/Redis wired like prod, multi-service compose, deployment parity	Docker / Compose

Neither replaces the other. The venv is your **default tool for "run Python on this machine"**; Docker is your **default tool for "run the same stack everywhere else."** If you skip the venv and only use Docker, every small test pays container startup cost; if you only use a venv, you can miss issues that only appear in the containerized layout—so both show up in a solid workflow.

---

1. Install the venv package (some Ubuntu installs ship without it).

```
sudo apt install -y python3-venv python3-pip
```

2. Make sure you're in your home directory.

```
cd ~
```

3. Create the project folder.

```
mkdir -p ~/docker-course/fastapi-app
```

4. Move into it.

```
cd ~/docker-course/fastapi-app
```

5. Confirm.

```
pwd
```

```
/home/your-user/docker-course/fastapi-app
```

6. Create a virtual environment in a folder called `.venv`. The dot prefix hides it from `ls` by default.

```
python3 -m venv .venv
```

(No output. Takes a couple of seconds.)

7. See what got created.

```
ls -la .venv
```

You should see `bin`, `include`, `lib`, `pyvenv.cfg`.

8. Activate the venv.

```
source .venv/bin/activate
```

9. Look at your prompt — it should now start with `(.venv)`. That's how you know the venv is active.

```
(.venv) your-user@host:~/docker-course/fastapi-app$
```

10. See which Python you're using now.

```
which python
```

```
/home/your-user/docker-course/fastapi-app/.venv/bin/python
```

(That's the **venv** Python, not the system one.)

11. Same for pip.

```
which pip
```

```
/home/your-user/docker-course/fastapi-app/.venv/bin/pip
```

12. Check the Python version inside the venv.

```
python --version
```

```
Python 3.10.12
```

(Note: you can now type `python`, not `python3` — the venv aliases it.)

13. Deactivate the venv.

```
deactivate
```

14. Look at your prompt — the `(.venv)` is gone. You're back to system Python.

```
which python
```

```
/usr/bin/python
```

(Or `python3` only — depends on your distro.)

15. Reactivate it. You'll be doing this a lot.

```
source .venv/bin/activate
```

16. Confirm the prompt is back to `(.venv)`.

```
which python
```

```
/home/your-user/docker-course/fastapi-app/.venv/bin/python
```

17. Add `.venv/` to a `.gitignore` so the venv never gets committed if this becomes a git repo.

```
echo ".venv/" > .gitignore
```

18. Confirm.

```
cat .gitignore
```

```
.venv/
```

19. Log it.

```
echo "venv created at ~/docker-course/fastapi-app/.venv" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **Every time you open a new terminal, the venv is NOT active.** You will have to run `source .venv/bin/activate` again. This is the #1 source of "why is FastAPI not found" errors for the rest of the module.
- The `(.venv)` prompt prefix is your single source of truth. If you don't see it, the venv is off.

- Don't run `pip install` outside the venv. It will install into your system Python and you'll waste an hour debugging.

Checkpoint

```
which python
```

You should see `/home/.../fastapi-app/.venv/bin/python`.

```
echo $VIRTUAL_ENV
```

You should see the path to your `.venv` directory.

```
ls -la ~/docker-course/fastapi-app
```

You should see `.venv` and `.gitignore`.

Next up: 02-installing-fastapi.md — Install FastAPI into your venv

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Install FastAPI and Uvicorn into the active venv and freeze the exact package versions into a `requirements.txt` with `pip freeze`.

Lesson 02: Installing FastAPI

By the end, you will have FastAPI and Uvicorn installed in your venv, and a `requirements.txt` file pinning your dependencies.

~10 minutes of typing

---

1. Make sure you're in the project directory and the venv is active. Look for `(.venv)` in your prompt.

```
cd ~/docker-course/fastapi-app
source .venv/bin/activate
```

2. Confirm pip points to the venv.

```
which pip
```

```
/home/your-user/docker-course/fastapi-app/.venv/bin/pip
```

3. Upgrade pip itself first.

```
pip install --upgrade pip
```

4. Install FastAPI and Uvicorn (the ASGI server FastAPI runs on). The `[standard]` adds extras like the watchfiles reloader.

```
pip install fastapi "uvicorn[standard]"
```

(You'll see several packages download.)

5. Confirm FastAPI is installed.

```
pip show fastapi | head -n 3
```

```
Name: fastapi
Version: 0.110.0
Summary: FastAPI framework, high performance, easy to learn, fast to code, ready
for production
```

(Version will vary.)

6. Confirm Uvicorn is installed.

```
pip show uvicorn | head -n 3
```

```
Name: uvicorn
Version: 0.29.0
...
```

7. See the full list of installed packages.

```
pip list
```

You'll see fastapi, uvicorn, and a dozen transitive dependencies (pydantic, starlette, etc.).

8. Save those exact versions to a `requirements.txt`. This is how you make a project reproducible.

```
pip freeze > requirements.txt
```

9. Read the file you just created.

```
cat requirements.txt
```

You should see lines like `fastapi==0.110.0`, `pydantic==2.6.4`, `uvicorn==0.29.0`, etc. Each package pinned to an exact version.

10. Count how many packages got installed.

```
wc -l requirements.txt
```

```
20 requirements.txt
```

(Number will vary, usually 15-25.)

11. Log it.

```
echo "FastAPI installed: $(pip show fastapi | grep Version)" >> ~/docker-course/notes/notes.txt
```

Watch out

- If pip says `Defaulting to user installation because normal site-packages is not writeable`, your venv is **not** active. Stop, run `source .venv/bin/activate`, and try again.
- Do not commit `.venv/` to git — that's why you wrote `.venv/` to `.gitignore` in lesson 01. Only `requirements.txt` should be committed.

Checkpoint

```
pip show fastapi | head -n 1
```

You should see `Name: fastapi`.

```
ls requirements.txt
```

You should see the path printed (no error).

```
grep fastapi requirements.txt
```

You should see a line like `fastapi==0.110.0`.

```
python -c "import fastapi; print(fastapi.__version__)"
```

You should see a version number.

Next up: 03-hello-world.md — Write your first FastAPI app

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Write a minimal FastAPI app in `main.py`, run it with `uvicorn main:app --reload`, and verify responses from curl, a browser, and the `/docs` Swagger UI.

Lesson 03: Hello World

By the end, you will have a running FastAPI server that you can hit from curl and from a browser, with hot-reload turned on.

~20 minutes of typing

> This is the "it's alive" lesson. Type slowly. Run the server after each meaningful change. The dopamine hit of seeing your code respond to a real HTTP request is the entire point.

---

1. Make sure you're in the project directory and venv is active.

```
cd ~/docker-course/fastapi-app
source .venv/bin/activate
```

2. Create an empty `main.py` file.

```
touch main.py
```

3. Open `main.py` in VS Code. From the terminal:

```
code main.py
```

(If `code` doesn't work, click `main.py` in VS Code's file explorer.)

4. Type the import line at the top.

```
from fastapi import FastAPI
```

5. Save (Ctrl+S). Then go back to the terminal and confirm Python can import the file with no errors.

```
python -c "import main"
```

(No output = no errors. If you see `ModuleNotFoundError: No module named 'fastapi'`, your venv isn't active.)

6. Add an empty line, then create the app instance. Type this into `main.py`:

```
app = FastAPI()
```

7. Save. Confirm the file looks like this.

```
cat main.py
```

```
from fastapi import FastAPI

app = FastAPI()
```

8. Add the first route. Type these four lines after `app = FastAPI()`:

```


@app.get("/")
def read_root():
    return {"hello": "world"}
```

9. Save. Confirm.

```
cat main.py
```

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def read_root():
    return {"hello": "world"}
```

10. Run the server with hot-reload.

```
uvicorn main:app --reload
```

You should see:

```
INFO:     Will watch for changes in these directories: [...]
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [...]
INFO:     Started server process [...]
INFO:     Application startup complete.
```

11. Leave that terminal running. Open a **second** integrated terminal in VS Code (the `+` button at the top-right of the terminal panel, or the dropdown). In the new terminal, hit your server.

```
curl http://localhost:8000
```

```
{"hello":"world"}
```

12. Hit it again, asking for a verbose response so you can see the headers.

```
curl -i http://localhost:8000
```

```
HTTP/1.1 200 OK
date: ...
server: uvicorn
content-length: 17
content-type: application/json

{"hello":"world"}
```

13. Open a browser and visit `http://localhost:8000`.

- **WSL students:** Windows browser, `http://localhost:8000`.
- **SSH students:** look at VS Code's "Ports" tab in the bottom panel — port 8000 should be auto-forwarded. Click the globe icon to open in your local browser.

You should see `{"hello":"world"}`.

14. Now visit `http://localhost:8000/docs` in the browser. **This is the FastAPI superpower.** You should see a Swagger UI page listing your `GET /` route.

15. Click on the `GET /` row → click the **"Try it out"** button → click **"Execute"**. Look at the response — same `{"hello":"world"}` you got from curl.

16. Now feel the hot-reload. Go back to `main.py` and change the response to:

```
    return {"hello": "fastapi"}
```

17. Save. Watch your first terminal — you should see uvicorn say:

```
WARNING:  WatchFiles detected changes in 'main.py'. Reloading...
INFO:     Application startup complete.
```

18. Curl it again — no need to restart anything.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

19. Refresh the browser at `/` — same change.

20. Stop the server. In the first terminal, press **Ctrl+C**.

```
INFO:     Shutting down
INFO:     Application shutdown complete.
```

21. Restart it.

```
uvicorn main:app --reload
```

22. Log it.

```
echo "FastAPI hello-world running on port 8000" >> ~/docker-course/notes/notes.txt
```

Watch out

- **`uvicorn main:app --reload`** — `main` is the file (without `.py`), `app` is the variable inside it. If you renamed the file to `app.py`, you'd run `uvicorn app:app --reload`.
- Without `--reload`, you'd have to stop and restart the server after every change. **Always use `--reload` during development.**
- "Address already in use" → another `uvicorn` is still running somewhere. `pkill uvicorn` or `lsof -i :8000` to find and kill it.
- Indentation matters in Python. The body of `read_root` must be indented 4 spaces (VS Code does this for you).

Checkpoint

```
curl http://localhost:8000
```

You should see `{"hello":"fastapi"}` (or `world` if you didn't change it back — either is fine).

In the browser at `http://localhost:8000/docs`, you should see Swagger UI listing one route: `GET /`.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000
```

You should see `200`.

```
cat main.py | wc -l
```

You should see at least 7 lines.

Next up: 04-path-and-query-params.md — Add routes that take parameters

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Define routes with typed path parameters and optional query parameters, and demonstrate FastAPI rejecting mistyped input with 422 validation errors.

Lesson 04: Path and Query Params

By the end, you will have a route that takes a typed path parameter and an optional query parameter, and you will have seen FastAPI's automatic validation reject bad input with a 422.

~25 minutes of typing

> Type hints in FastAPI aren't decoration — they're validation. The framework reads them at startup and will reject any request whose params don't match.

---

1. Make sure your venv is active and uvicorn is still running with `--reload`.

```
cd ~/docker-course/fastapi-app && source .venv/bin/activate
```

(Skip if you already see `(.venv)` in your prompt.)

2. Open `main.py` in VS Code.

3. Add a new route below the hello-world one. Type:

```


@app.get("/items/{item_id}")
def read_item(item_id):
    return {"item_id": item_id}
```

4. Save. Watch uvicorn auto-reload in the first terminal.

5. Curl with a number.

```
curl http://localhost:8000/items/42
```

```
{"item_id":"42"}
```

(Note: it came back as a **string** because `item_id` has no type hint.)

6. Curl with text.

```
curl http://localhost:8000/items/banana
```

```
{"item_id":"banana"}
```

(Also accepted — no validation yet.)

7. Now add a type hint. Change the function signature to:

```
def read_item(item_id: int):
```

8. Save. Curl with a number.

```
curl http://localhost:8000/items/42
```

```
{"item_id":42}
```

(No quotes around 42 — it's an int now.)

9. Curl with text.

```
curl http://localhost:8000/items/banana
```

```
{"detail":[{"type":"int_parsing","loc":["path","item_id"],"msg":"Input should be a
valid integer..."}]}
```

FastAPI rejected it with a 422. This is the validation in action.

10. Confirm the status code.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/banana
```

```
422
```

11. Now add an optional query parameter. Change the function signature to:

```
def read_item(item_id: int, q: str | None = None):
    return {"item_id": item_id, "q": q}
```

12. Save.

13. Curl without `q`.

```
curl "http://localhost:8000/items/42"
```

```
{"item_id":42,"q":null}
```

14. Curl with `q`. **Quote the URL** — the shell will eat the `?` otherwise.

```
curl "http://localhost:8000/items/42?q=hello"
```

```
{"item_id":42,"q":"hello"}
```

15. Try multiple query params.

```
curl "http://localhost:8000/items/42?q=hello&extra=ignored"
```

```
{"item_id":42,"q":"hello"}
```

(`extra` is silently ignored — it's not declared.)

16. Open `http://localhost:8000/docs` in the browser. You should now see two routes: `GET /` and `GET /items/{item_id}`.

17. Click `GET /items/{item_id}` → **"Try it out"**. Notice Swagger now shows `item_id` as required and `q` as optional, with the right types.

18. Type `42` for `item_id`, leave `q` empty, click **Execute**. Same response as the curl.

19. Type `banana` for `item_id` and click Execute. Swagger shows the 422 with the readable error message.

20. Add another query param with a default value. Change the signature to:

```
def read_item(item_id: int, q: str | None = None, short: bool = False):
```

21. Update the function body to use it:

```
    if short:
        return {"item_id": item_id}
    return {"item_id": item_id, "q": q}
```

22. Save. Curl without `short`.

```
curl "http://localhost:8000/items/42?q=hi"
```

```
{"item_id":42,"q":"hi"}
```

23. Curl with `short=true`.

```
curl "http://localhost:8000/items/42?q=hi&short=true"
```

```
{"item_id":42}
```

24. Try `short=banana` — booleans validate too.

```
curl "http://localhost:8000/items/42?short=banana"
```

```
{"detail":[{"type":"bool_parsing","loc":["query","short"],"msg":"Input should be a valid boolean..."}]}
```

25. Confirm the file.

```
cat main.py
```

You should see the hello-world route, the items route, and three params on the items function.

26. Log it.

```
echo "Path + query params + type validation: working" >> ~/docker-course/notes/notes.txt
```

Watch out

- Always **quote the URL** when curling with query params: `curl "...?q=foo"`. Without quotes, your shell tries to interpret `?` and `&`.
- `str | None` is Python 3.10+ syntax. If you're on 3.9 or older, use `Optional[str]` (and `from typing import Optional` at the top). The lessons assume 3.10+.
- The default value is what makes a param optional. `q: str` requires the query param. `q: str | None = None` makes it optional.
- A 422 response is **not** a server crash. FastAPI is doing its job by rejecting bad input. The right response is to fix the request.

Checkpoint

```
curl -s "http://localhost:8000/items/42?q=test&short=false"
```

You should see `{"item_id":42,"q":"test"}`.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/banana
```

You should see `422`.

```
curl -s -o /dev/null -w "%{http_code}\n" "http://localhost:8000/items/42?
short=true"
```

You should see `200`.

In the browser at `/docs`, the `/items/{item_id}` route should show three params: `item_id` (path, required), `q` (query, optional, string), `short` (query, optional, boolean).

Next up: 05-request-bodies-with-pydantic.md — Accept JSON bodies and validate them with Pydantic

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.